DOMAIN- MICROARCHITECTURE DESIGN

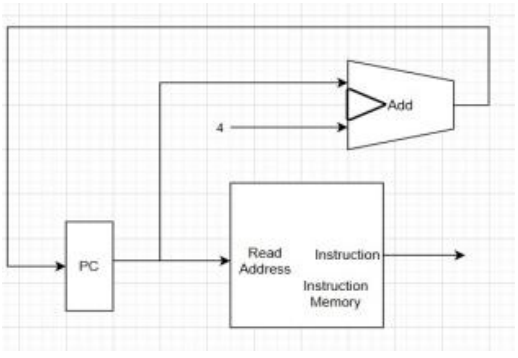
NUMBER 1

The Instruction Set Architecture (ISA) that underpins this microarchitecture is a custom RISC-based design developed to enable efficient, low-cost processing of cough sounds for tuberculosis (TB) detection. It is optimized for mobile and embedded devices where power efficiency and simplicity are critical. The ISA follows a Reduced Instruction Set Computer (RISC) philosophy, using fixed 32-bit instruction formats that simplify decoding and pipelining while ensuring consistent performance. It features 16 general-purpose 32-bit registers and supports 8-bit, 16-bit, and 32-bit data types, which are common in audio signal processing. The design includes multiple addressing modes (immediate, register indirect, and base with offset) and a little-endian, word-aligned memory model for easy hardware implementation. Its instruction set covers arithmetic, logical, data movement, immediate, branching, and specialized signal processing operations—such as squaring, absolute difference, and feature comparison tailored for analyzing cough patterns. The ISA prioritizes simplicity, speed, and low power consumption over code density or advanced capability, ensuring that it performs real-time signal processing efficiently on resource-limited systems. Moreover, it is extensible, allowing future integration of advanced AI or audio-processing instructions (e.g., FFT or MFCC) to enhance its analytical accuracy without requiring a complete redesign.

Supported Instruction Classes	R-TYPE, I-TYPE, B-TYPE, AND MEMORY INSTRUCTIONS.
Design objective	To implement simple pipe-lined data-path that is used for AI based devices.

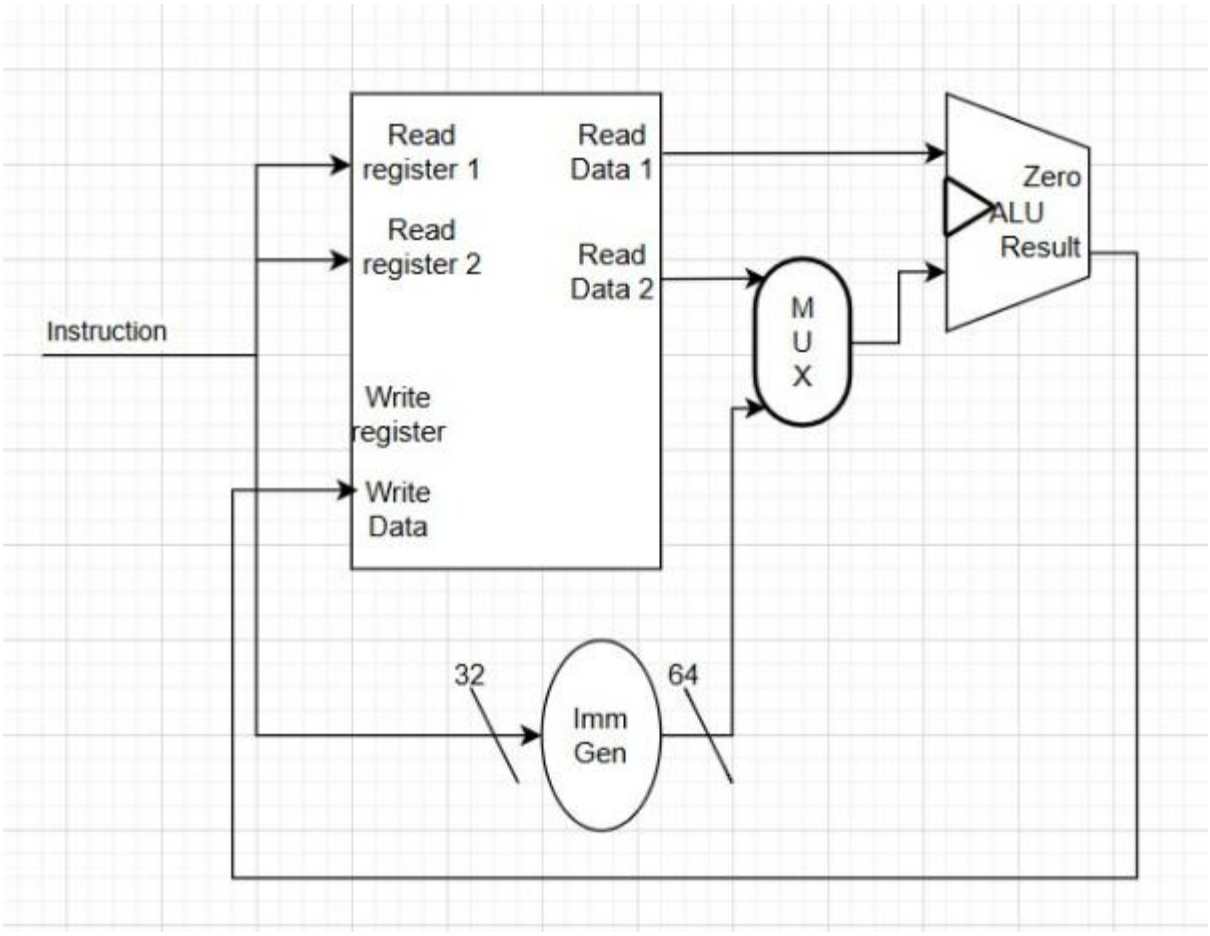
NUMBER 2

INSTRUCTION FETCH



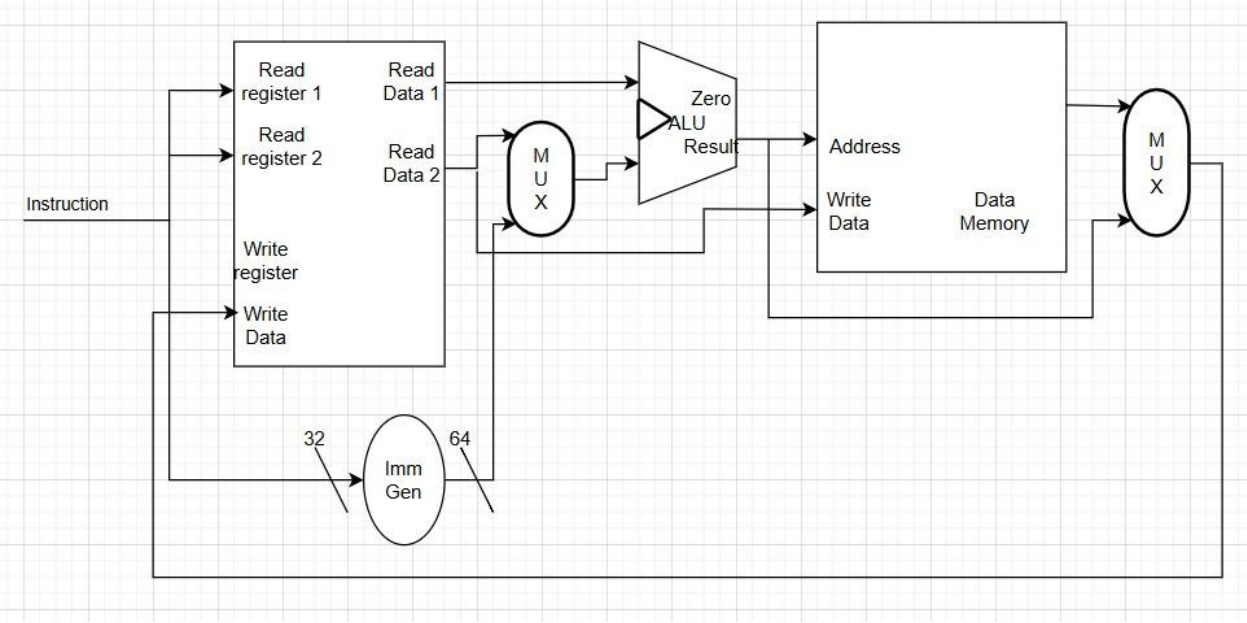
A data-path that fetches instructions and increments the PC to obtain the address of the next sequential instructions. To execute any instruction, we need a memory unit to store instructions of a program and supply instructions given an address. A program counter which is a register that holds address of the current instruction. Lastly we will need an adder to increment the PC by 4 bytes of the next instruction. The supply instruction is used by other parts of the data path. We assume that for every class of instructions, the instruction fetch is already present

R-TYPE + I-TYPE INSTRUCTIONS



The I-format and R-format both use the ALU and the register file, the R-format has two source registers to be read from the register file and then the required operation is performed on their values by the ALU, and then the result is written back to the destination register. But for the I-type instructions, the second input of the ALU is the 32-bit offset field from the instruction, and the first input is the source register. But the ALU requires the 64-bit input, hence the immediate generator is included to sign extend the 32-bit offset to 64-bit. The multiplexer is used to select the second operand of the ALU which can be a source register for R-type and the immediate value for the I-type.

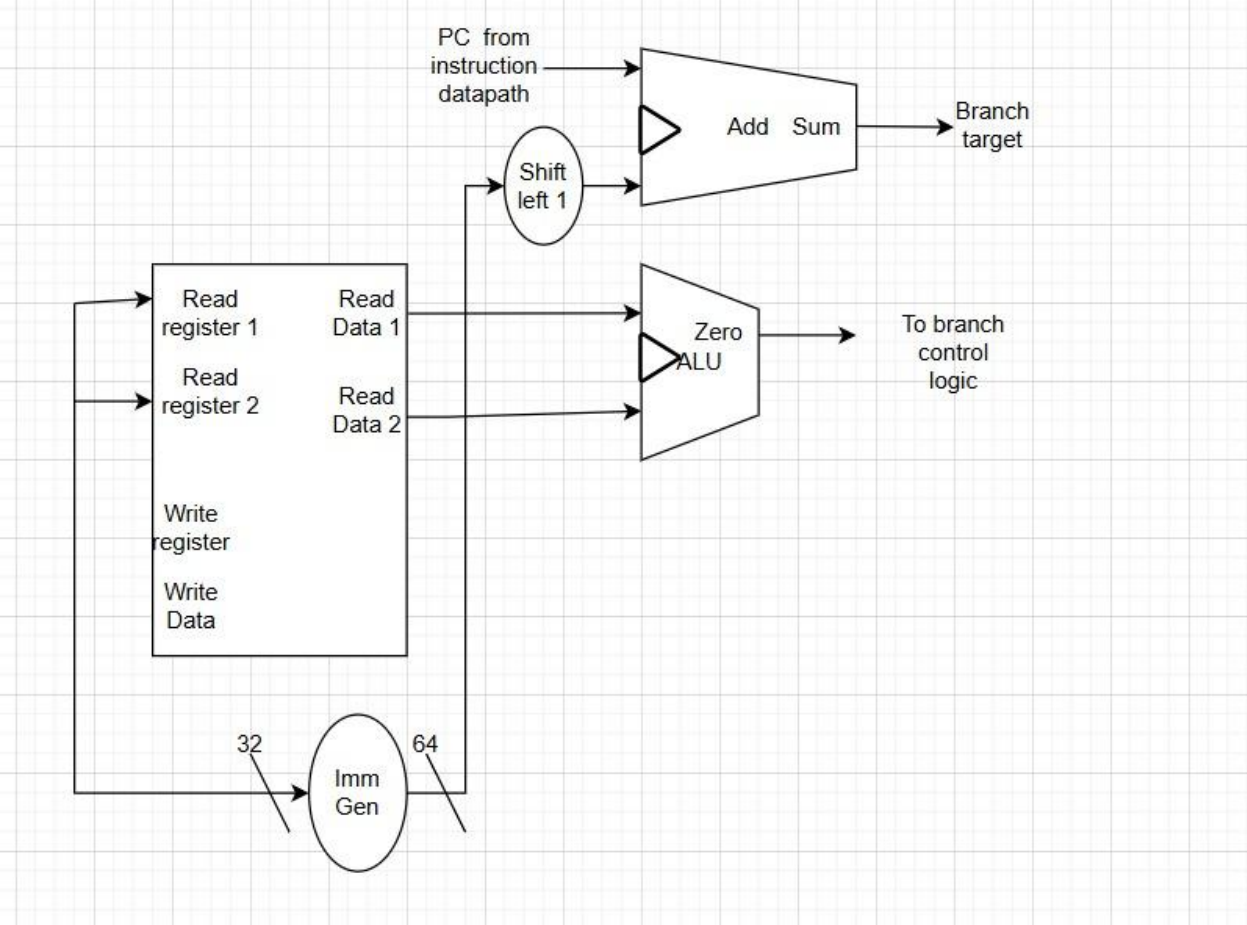
LOAD/STORE + I-TYPE + R-TYPE INSTRUCTIONS



The R-type instructions, I-type, and memory instructions all use the ALU, and the register file. But for the I-type and the memory instructions also use the immediate generator. And only memory instructions write (for store) or read (for load) from memory. For I-type and memory instructions, the second input of the ALU is the sign extended immediate value from the instruction, while for R-type, the second operand is the second source register. So the first multiplexer helps to select the

second input of the ALU depending on the instruction class being executed. For I-type and R-type, the value written to the destination register in the register file is directly from the ALU. But for the load memory instruction, the input into the register file is read from the memory, but the store instruction does not write into the register file, but it writes into the memory. So the second multiplexer selects the where to read the value that is written to the register file (either from the ALU or memory), depending on the class of instruction being executed.

BRANCH INSTRUCTIONS

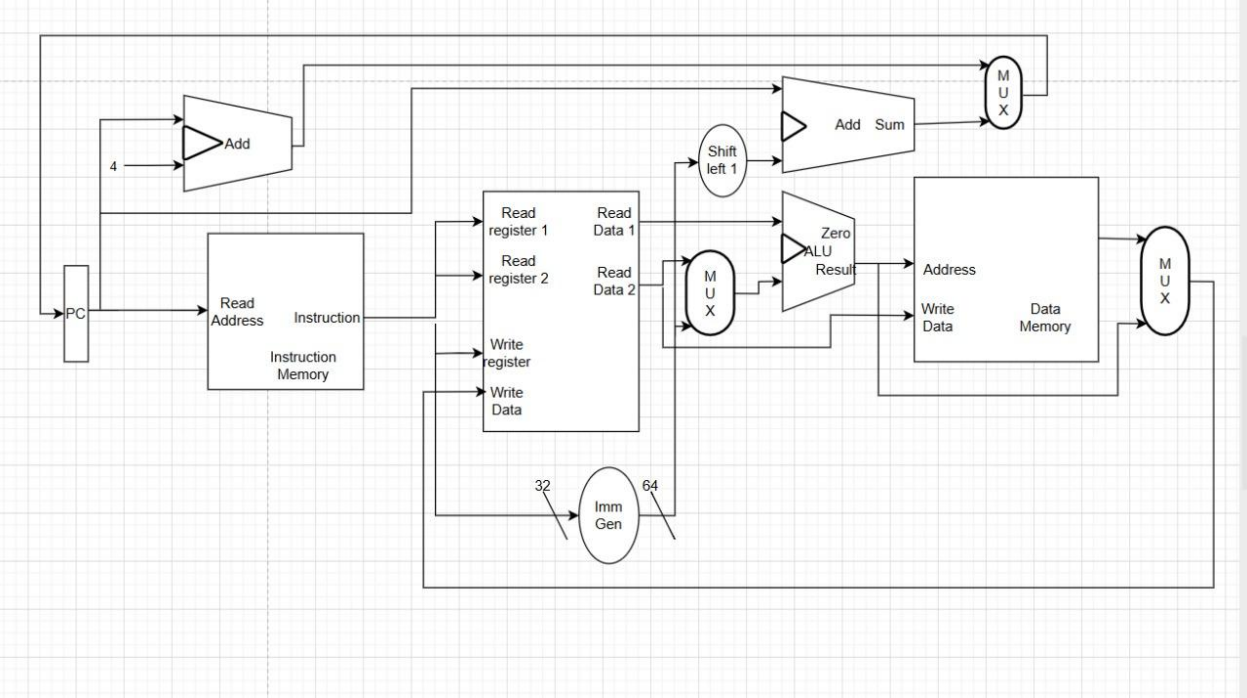


After instruction decode stage, the branch instruction reads two source registers from the register file and supplies the immediate offset value which is used to calculate the branch target address which is the address of the instruction to be

executed provided the branch condition is true. This offset is sign extended by the immediate generator, and then shifted left by 1 so that it is a double-word shift. After shifting, the offset is added to the PC using an adder so as to calculate the branch target address. The condition can be evaluated by subtracting the values in the two source registers using ALU, because when the values in the two source registers are equal, the result becomes zero, otherwise, the values are not equal. And whenever the result of the ALU is zero, the ALU provides the signal which can then be used to implement the branch logic, to decide whether the next instruction is in PC + 4 or in branch target address.

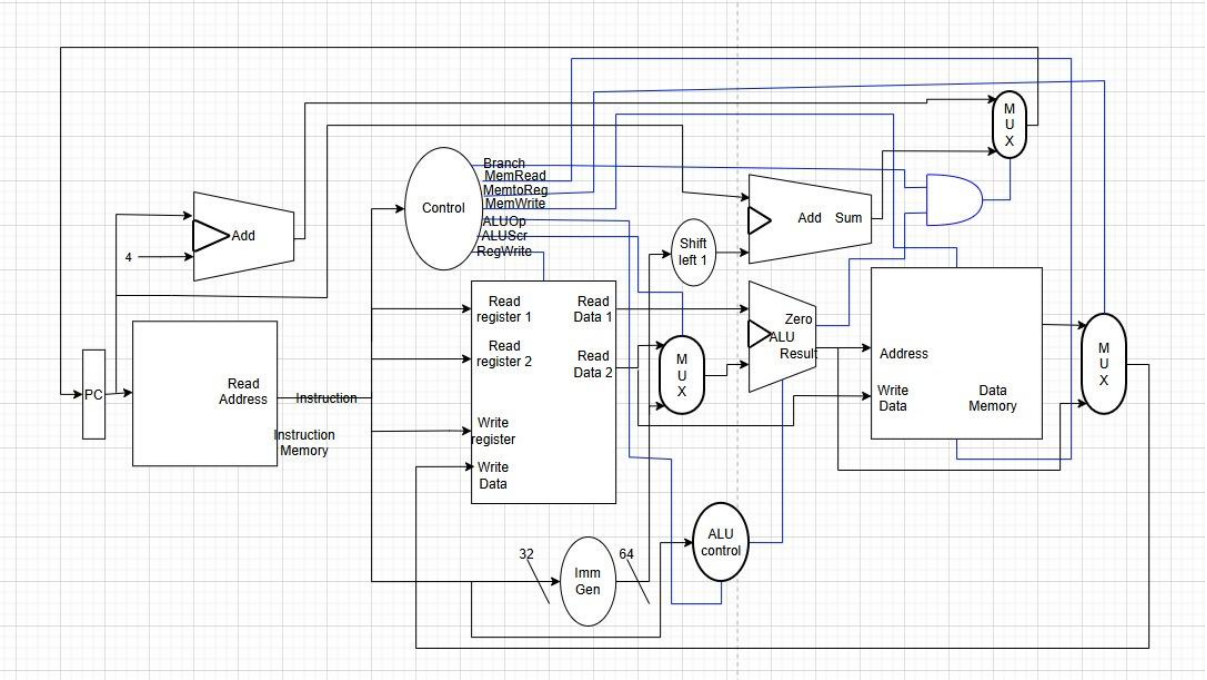
NUMBER 3

UNIFIED DATA-PATH



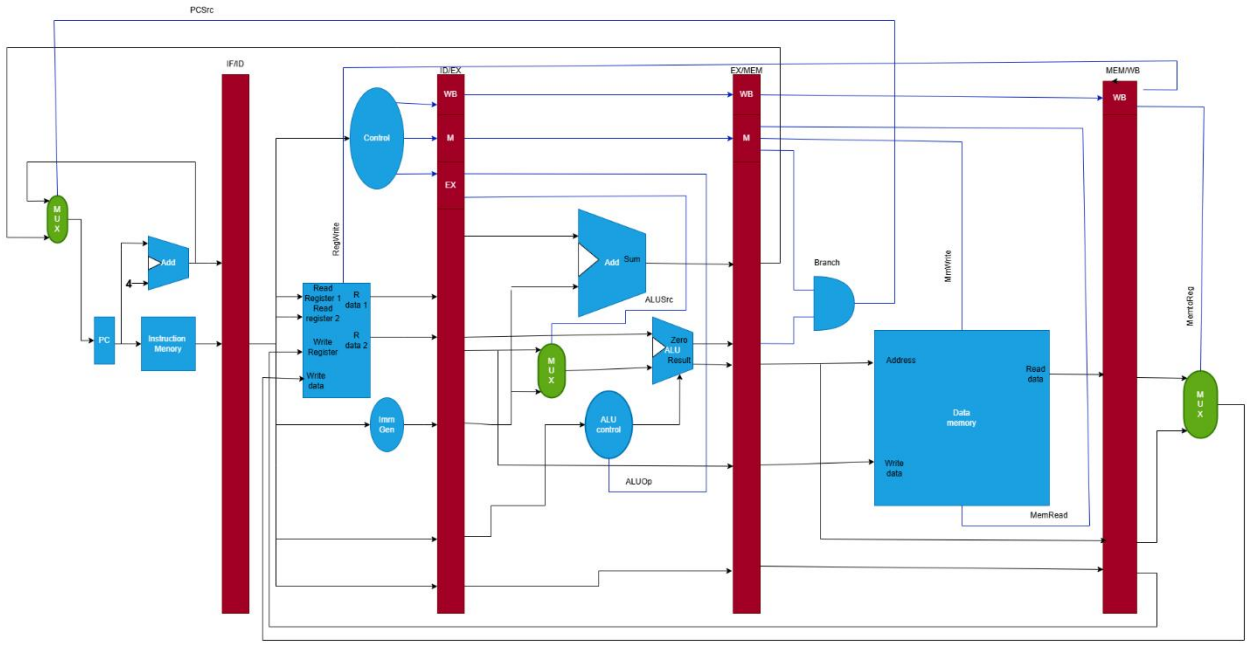
NUMBER 4

UNIFIED DATA-PATH AND CONTROL UNIT



Instruction opcode	**ALU Op**	**Operation**	**Funct7 Field**	**Desired ALU action**	**Funct3 field**	**ALU control input**
ld	00	load doubleword	xxxxxxx	Add	xxx	0010
sd	00	Store doubleword	xxxxxxx	Add	xxx	0010
Branch equal	01	Branch if equal	xxxxxxx	Subtract	xxx	0110
R-type	10	add	0000000	Add	000	0010
R-type	10	Sub	0100000	Subtract	000	0110
R-type	10	AND	0000000	AND	111	0000
R-type	10	OR	0000000	OR	110	0001

NUMBER 5



Pipeline Hazards and pipeline Optimization

Structural hazards

When a planned instruction cannot execute in a proper clock cycle because the hardware does not support the combination of instructions that are set to execute

Data Hazard

This is when a planned instruction cannot execute in a proper clock cycle because the data that is needed for execution is not available. For example, suppose we have an add instruction that is followed immediately by a subtract instruction that uses the sum. The add instruction does not write its result until the fifth stage, meaning we would have to waste three clock cycles in the pipeline.

Load-Use Data Hazard

This is a specific form of data hazard in which the data being loaded by the load instruction is not yet available when needed by some other instruction.

Control Hazards (branch hazard)

This is when an instruction cannot be executed properly in the pipeline clock cycle because the instruction that was fetched is not the one that is needed. That is, the flow of instruction address is not what the pipeline expected.

Pipeline Stall

This is also referred to as a bubble. It is a stall initiated in order to resolve a data hazard.

Forwarding

This is a technique used to resolve data hazards by sending the result of an instruction directly to the input of the ALU of the subsequent instruction without waiting for the result to be written back to register file.

References:

Patterson, D.A. and Hennessy, J.L., 2021. Computer organization and design: The hardware/software interface (RISC-V edition). Burlington, MA:
Morgan Kaufmann.